

Pillar 2: Advanced React & Next.js Ecosystem

Senior Frontend / Fintech interview answers. Each question includes a practical architecture-focused response with examples and trade-offs where relevant.

1. How do you implement a Lexical Editor-based rich text experience?

I build Lexical as an extensible editor platform with configuration, custom nodes, plugins, toolbar actions, markdown import/export, serialization, validation, and read-only rendering. I separate editing from rendering so saved content can be displayed safely in dashboards or AI assistant responses. I also handle permissions, sanitization, autosave, keyboard shortcuts, and tests for custom nodes.

2. Explain the benefits of Next.js App Router for complex navigation in banking.

The App Router supports nested layouts, route groups, loading states, server components, and cleaner route organization. In a banking portal, persistent layouts are useful for sidebars, headers, tenant selectors, and user context. Route-level loading and error boundaries also improve resilience. It helps organize complex areas like accounts, payments, risk, and admin under predictable route structures.

3. How do you optimize GraphQL queries using Apollo Client's cache?

I use normalized caching, typePolicies, cache keys, fragments, pagination policies, and correct fetch policies. Stable object IDs allow Apollo to reuse cached entities. I avoid over-fetching by designing focused queries and reusable fragments. After mutations, I update the cache or refetch only what is necessary. For frequently changing data, cache-and-network or subscriptions may be better than cache-first.

4. Describe the usage of React.memo and useMemo in high-frequency trading UIs.

React.memo helps prevent unnecessary child re-renders when props are stable. useMemo caches expensive calculations such as totals, grouped rows, or derived risk values. In high-frequency UIs, I also use virtualization, throttling, and stable callbacks. However, memoization is not free, so I measure first and use it only where renders or calculations are actually expensive.

5. How do you handle complex forms and validation in financial origination flows?

I use React Hook Form with Zod or Yup schemas, step-based form state, field-level validation, server-side validation, and clear error messages. Frontend validation improves UX, but backend validation remains mandatory. For multi-step loan or origination flows, I persist draft state, validate each step, support conditional fields, and handle document upload and review states.

6. What is your strategy for Global State vs Server State?

Server state includes API data, loading, errors, caching, and synchronization, so I manage it with Apollo Client, React Query, or SWR. Global client state includes theme, locale, auth user, tenant, feature flags, and permissions. Local UI state stays inside components where possible. This separation avoids overusing Redux for everything and makes data flows clearer.

7. How do you implement Error Boundaries to ensure graceful failure in production?

I wrap the app and risky sections with Error Boundaries. A global boundary catches unexpected crashes, while widget-level boundaries prevent one dashboard card from breaking the whole page. The fallback should be user-friendly and safe, such as 'Risk Exposure could not be loaded'. I also log the error with release, route, and user-safe context.

8. Describe how you've used React Native to share logic between web and mobile.

I share business logic, validation schemas, API clients, formatters, permission checks, constants, and TypeScript types. I keep UI components, navigation, storage, and device APIs platform-specific. For example, loan eligibility or transaction formatting can be shared, while the web and mobile screen layouts remain different. This improves consistency without forcing unnatural component reuse.

9. How do you manage Side Effects in Redux Saga for transaction workflows?

Redux Saga is useful for complex workflows such as transfer submission, OTP generation, OTP verification, transaction status polling, and cancellation. I model the workflow as a sequence of effects and handle retries, errors, and cancellation explicitly. For money movement, I avoid unsafe automatic retries unless the backend supports idempotency keys.

10. What are the pros/cons of Server Components for data-heavy investment intelligence?

Server Components reduce client JavaScript and can fetch data closer to the server. They work well for static or semi-static sections, report summaries, and data that does not need heavy client interaction. The drawbacks are a more complex mental model, careful server/client boundaries, and challenges with personalization or real-time widgets. For dashboards, I combine Server Components with Client Components for charts and filters.

11. How do you build higher-order components for permission-based UI?

A permission HOC wraps a component and renders it only if the user has the required permission. It keeps access control consistent across buttons, panels, and routes. In modern React I may prefer hooks or wrapper components, but HOCs are still useful when adding permission logic to many existing components. Backend authorization must still enforce real security.

12. Explain the reconciliation process in React and how it impacts performance.

Reconciliation is React's process of comparing the previous element tree with the next one and updating the DOM where necessary. Stable keys help React preserve list items correctly. Poor keys, unnecessary state changes, and large component trees can cause expensive renders. I improve performance by splitting components, using stable keys, memoizing carefully, and virtualizing large lists.

13. How do you handle localization in a multi-region banking app?

I avoid hardcoded strings and use translation keys. I handle text, date formats, number formats, currency, pluralization, RTL layout, and region-specific compliance copy. For financial apps, locale-sensitive formatting is critical because decimal separators, currency symbols, and date formats vary by market. I also test layout expansion for longer translations.

14. Describe a time you built a custom hook to simplify complex financial logic.

A good example is a hook such as `useRiskFilters` or `usePortfolioSelection`. It can centralize selected filters, default values, URL synchronization, reset behavior, and derived query variables. This removes repeated logic from multiple dashboard components and makes behavior easier to test. The hook should expose a small clear API instead of leaking internal state structure.

15. How do you optimize Next.js images and assets for faster initial load?

I use `next/image` where appropriate, compress images, serve modern formats, lazy load below-the-fold assets, and preload only critical resources. I avoid loading heavy chart libraries or admin modules on the initial route. For public banking pages, I optimize hero assets and fonts carefully to improve LCP and reduce layout shift.

16. What is your approach to React Context API vs Redux Toolkit?

I use Context for low-frequency global values such as theme, locale, and auth metadata. I use Redux Toolkit when state transitions are complex, many components need the same state, or debugging with DevTools is important. Context is simple but can cause unnecessary re-renders if used for frequently changing data. Server state should usually stay in Apollo, React Query, or SWR.

17. How do you handle Prop Drilling in a deeply nested banking dashboard?

I first check whether the component structure can be improved through composition. If data is needed deeply and broadly, I use Context, a custom hook, or a store. I also move state closer to where it is used. The goal is to avoid passing `selectedAccount` or `permissions` through many layers while also avoiding unnecessary global state.

18. Explain hydration in Next.js and how to avoid hydration mismatch errors.

Hydration is when React attaches event handlers to server-rendered HTML. Mismatches happen when the server output differs from the client's first render. I avoid `Date.now()`, `Math.random()`, browser-only APIs, and user-specific client values during SSR output. Browser-only code should run in `useEffect` or be guarded carefully. Initial server and client state must match.

19. How do you use Layouts in Next.js to persist UI across route changes?

I use layouts for shared UI such as sidebars, headers, breadcrumbs, and tenant switchers. In the App Router, nested layouts allow dashboard chrome to persist while page content changes. This improves performance and user experience because common UI does not remount unnecessarily. It also keeps route organization clean.

20. How do you implement Deep Linking in a React Native banking app?

I configure URL schemes and route mapping such as `bankapp://transactions/123` or `bankapp://cards/replace`. The app validates authentication and permissions before opening sensitive screens. Deep links should never bypass security. If the user is logged out, the app should authenticate first and then navigate to the intended destination safely.

21. Describe your experience with Kendo UI for enterprise-grade tables.

Kendo UI is useful for enterprise grids because it provides sorting, filtering, grouping, pagination, virtualization, export, and accessibility features. In financial apps, I customize cells for currency, risk status, permissions, loading states, and row actions. I still watch bundle size and avoid importing more modules than needed.

22. How do you ensure Type Safety in React component props?

I define explicit prop types, avoid `any`, use union types for variants, and use generics for reusable components. For example, a `Select` component can accept generic option types and typed `getLabel/getValue` callbacks. In a design system, type-safe props prevent invalid variants, sizes, and states from spreading across many apps.

23. What is the impact of React 18 Concurrent Mode on user experience?

React 18 introduced concurrent rendering capabilities that help React keep the UI responsive by preparing updates without blocking the main thread as aggressively. Features like transitions are useful when filtering large dashboards or updating expensive views. It does not automatically make every app faster; developers still need to avoid heavy synchronous work and optimize rendering.

24. How do you handle API rate limiting or retry logic in the frontend?

I use exponential backoff, retry limits, user-friendly error states, and retry only for safe operations. GET requests can often be retried, but transaction or payment submissions should not be blindly retried unless the backend supports idempotency. I also respect `Retry-After` headers, show clear messages, and avoid creating retry storms.

25. Describe how you've used Vite to improve development speed over Webpack.

Vite improves development speed by using native ES modules during dev and fast HMR. Startup and updates are typically faster because the whole app does not need to be bundled upfront. For React TypeScript apps, Vite usually reduces configuration complexity and improves feedback loops. Webpack is still powerful, but Vite is excellent for modern apps and libraries.